

M3 Math Note

Chung-En (Johnny) Yu

Department of Intelligent Systems and Robotics, University of West Florida

Tip

This math note serves as supplementary content for the lecture. Students are encouraged to read it in conjunction with the lecture.

Logistic Regression

Logistic regression is a widely used probabilistic model for binary classification tasks, where the goal is to classify inputs into one of two classes (e.g., "yes" or "no," "positive" or "negative"). It models the relationship between the input features and the probability of the positive class.

Understanding odds and log-odds

What are odds?

Odds represent the ratio of the probability of an event happening to the probability of it not happening. It is calculated as:

$$\text{Odds} = \frac{p}{1 - p}$$

Where:

- p is the probability of the event occurring.
- $1 - p$ is the probability of the event not occurring.

Exercise: If the probability of passing an exam (p) is 0.75 (75%), the odds are:

$$\text{Odds} = \frac{0.75}{1 - 0.75} = \frac{0.75}{0.25} = 3$$

This means the student is 3 times more likely to pass than to fail.

What are log-odds?

Log-odds, or the logarithm of the odds, is also known as the **logit** function, which transforms the odds into a continuous range that spans all real numbers, making it easier to model with linear functions. It is calculated as:

$$\text{Log-Odds} = \text{logit}(p) = \log\left(\frac{p}{1 - p}\right)$$

The log-odds range from $-\infty$ (as $p \rightarrow 0$) to $+\infty$ (as $p \rightarrow 1$).

Relating odds, log-odds, and probability

1. Start with the odds: $\text{Odds} = \frac{p}{1-p}$
2. Take the natural logarithm to get the log-odds: $\text{Log-Odds} = \log\left(\frac{p}{1-p}\right)$
3. Rearrange to express p in terms of log-odds: $p = \frac{1}{1+e^{-\text{Log-Odds}}}$

This final step demonstrates that the **sigmoid function** is used to map log-odds back to probabilities. (We will see it later when we tie it back to logistic regression model.)

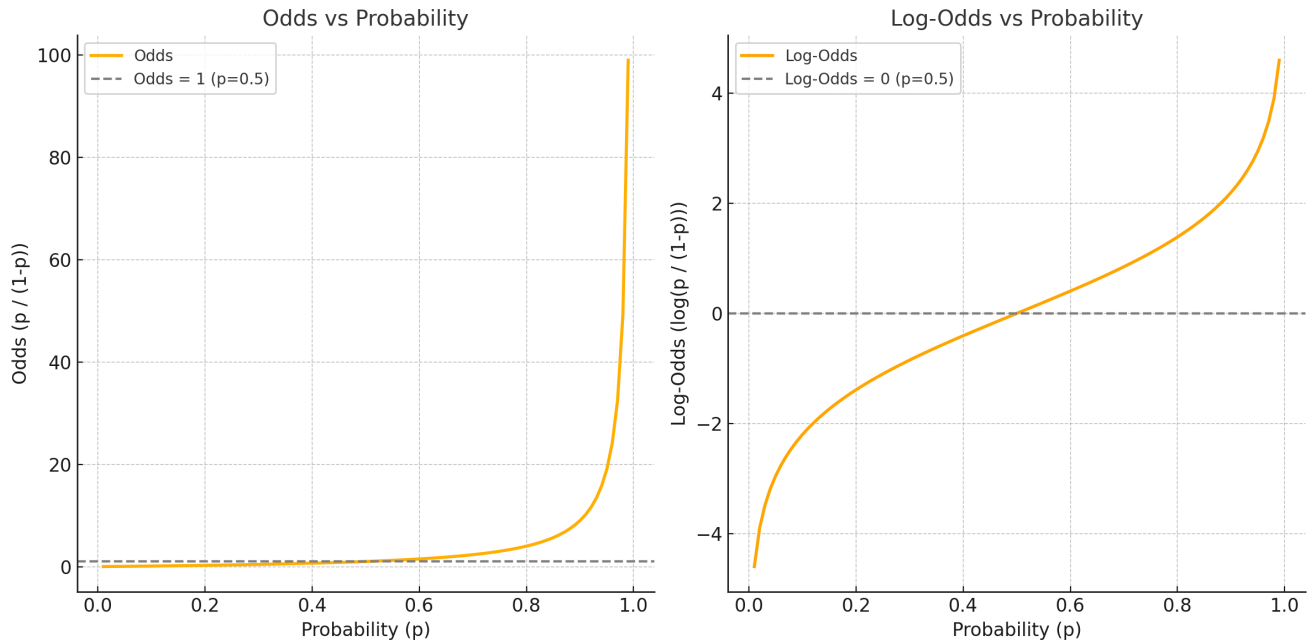
Examples of odds and log-odds

Imagine predicting whether it will rain tomorrow ($y = 1$) based on atmospheric conditions. If the model estimates $p = 0.8$ (80% probability of rain). Let's calculate the odds and log-odds for various probabilities:

- If $p = 0.2$ (20% chance):
 - $\text{Odds} = \frac{0.2}{1-0.2} = \frac{0.2}{0.8} = 0.25$
 - $\text{Log-Odds} = \log(0.25) \approx -1.39$
- If $p = 0.5$ (50% chance):
 - $\text{Odds} = \frac{0.5}{1-0.5} = 1$
 - $\text{Log-Odds} = \log(1) = 0$
- If $p = 0.8$ (80% chance):
 - $\text{Odds} = \frac{0.8}{1-0.8} = 4$
 - $\text{Log-Odds} = \log(4) \approx 1.39$

Observation:

- When $p = 0.5$, the odds are 1 (equal chances), and the log-odds are 0.
- As $p \rightarrow 1$, the odds and log-odds increase, reflecting higher likelihood, i.e., higher chance will rain tomorrow.
- As $p \rightarrow 0$, the odds and log-odds decrease, reflecting lower likelihood, i.e., lower chance will rain tomorrow.



Using log-odds in logistic regression

Logistic regression models the log-odds as a linear function of the input features ($w^T x + b$). Thus, in the context of logistic regression, log-odds is more often to be called logit function. This linear relationship makes it possible to apply optimization techniques like gradient descent to find the model parameters.

Under the logistic model, we assume that there is a linear relationship between the weighted inputs (net input z), so the log-odds can be written as:

$$z = \text{logit}(p) = w_1 x_1 + \dots + w_m x_m + b = \sum_{i=1}^m w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Where,

- $w_1, w_2, \dots, w_m$ are the weights for the input features.
- b is the bias term.
- $x_1, x_2, \dots, x_m$ are the input features.

The logit function map the probability p (class membership probability of an example given its feature) to a real-number range, we can use the inverse function to revert the real range to $[0, 1]$ range for the probability p . The inverse of the logit function called the **logistic sigmoid function** or **sigmoid function**:

$$\text{sigmoid function } \sigma(z) = \frac{1}{1 + e^{-z}} \quad , \text{ net input } z = \mathbf{w}^T \mathbf{x} + b$$

In Adaline, we used the identity function, $\sigma(z) = z$, as the activation function. In logistic regression, this activation function simply becomes a sigmoid function.

Exercise: Suppose you're building a model to predict if a student passes an exam ($y = 1$) based on study hours (x_1) and attendance rate (x_2). The model's prediction for p , the probability of passing, might look like this:

$$p = \frac{1}{1 + e^{-(0.5x_1 + 0.3x_2 - 1.2)}}$$

Here, $w_1 = 0.5$, $w_2 = 0.3$, and $b = -1.2$. Adjusting x_1 or x_2 will change p , illustrating the relationship between features and the predicted probability.

Cost function of logistic regression

In logistic regression, the cost function measures the error between predicted probabilities and actual outcomes. It is derived from the likelihood of the data and is expressed as the **negative log-likelihood**:

$$L(\mathbf{w}, b \mid \mathbf{x}) = \sum_{i=1}^n \left[-y^{(i)} \log \left(\sigma \left(z^{(i)} \right) \right) - \left(1 - y^{(i)} \right) \log \left(1 - \sigma \left(z^{(i)} \right) \right) \right]$$

Where,

- n is the number of training samples.
- $y^{(i)}$ is the true label for the i -th sample.
- $\sigma(z^{(i)})$ is the predicted probability for the i -th sample.

This function penalizes incorrect predictions more heavily as their probability diverges from the true label. The goal of training a logistic regression model is to find the parameter values that minimize the cost function.

Before we move on, let's find out how we obtain the cost function.

Deriving the cost function

Given that we have a binary classification problem with class labels 0 and 1, we can model the label as Bernoulli variable, with the probability p being 1: $Y \sim \text{Bern}(p)$. We can express the probability of the label being 1 or 0 given the input (a single data point):

$$\begin{aligned} P(Y = 1 \mid X = x^{(i)}) &= \sigma(z^{(i)}) \\ P(Y = 0 \mid X = x^{(i)}) &= 1 - \sigma(z^{(i)}) \end{aligned}$$

Putting these two expressions together, we get the probability mass function of the Bernoulli variable:

$$P(Y = y^{(i)} \mid X = x^{(i)}) = p(y^{(i)} \mid x^{(i)}) = \left(\sigma(z^{(i)}) \right)^{y^{(i)}} \left(1 - \sigma(z^{(i)}) \right)^{1-y^{(i)}}$$

Given that all training examples are independent, the probability of all events occurring is the likelihood of the training labels. We can express the likelihood of the training labels as:

$$\mathcal{L}(\mathbf{w}, b \mid \mathbf{x}) = \prod_{i=1}^n p(y^{(i)} \mid \mathbf{x}^{(i)}; \mathbf{w}, b)$$

Then substituting the probability mass function of the Bernoulli variable to express the likelihood:

$$\mathcal{L}(\mathbf{w}, b \mid \mathbf{x}) = \prod_{i=1}^n \left(\sigma(z^{(i)}) \right)^{y^{(i)}} \left(1 - \sigma(z^{(i)}) \right)^{1-y^{(i)}}$$

With such expression, we may use logarithm to convert the problem of maximizing the likelihood function into the equivalent problem of minimizing the negative log-likelihood function.

- Maximizing $\mathcal{L}(\mathbf{w}, \mathbf{b} \mid \mathbf{x}) = \text{Minimizing } -\log(\mathcal{L}(\mathbf{w}, \mathbf{b} \mid \mathbf{x}))$
- **Loss** is now defined as: $-\log(\mathcal{L}(\mathbf{w}, \mathbf{b} \mid \mathbf{x}))$

Thus, let's rewrite the cost function after applying negative logarithm:

$$L(\mathbf{w}, \mathbf{b} \mid \mathbf{x}) = \sum_{i=1}^n \left[-y^{(i)} \log \left(\sigma \left(z^{(i)} \right) \right) - \left(1 - y^{(i)} \right) \log \left(1 - \sigma \left(z^{(i)} \right) \right) \right]$$

The cost function can then be used as the objective function in an optimization algorithm like gradient descent to minimize the loss.

Updating logistic regression parameters via gradient descent

Compare to Adaline, the weights and bias update rules in logistic regression remained unchanged. We can show it as calculating the partial derivative of the log-likelihood function with respect to the j -th weight:

$$\frac{\partial L}{\partial w_j} = \underbrace{\frac{\partial L}{\partial a} \frac{da}{dz} \frac{\partial z}{\partial w_j}}_{\text{Apply chain rule}} \quad \text{where } a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

1) Derive terms separately:

2) Combine via chain rule and simplify:

$$\begin{array}{l} \left. \begin{array}{l} \frac{\partial L}{\partial a} = \frac{a - y}{a \cdot (1 - a)} \\ \frac{da}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2} = a \cdot (1 - a) \end{array} \right\} \longrightarrow \frac{\partial L}{\partial z} = a - y \\ \left. \begin{array}{l} \frac{\partial z}{\partial w_j} = x_j \end{array} \right\} \longrightarrow \frac{\partial L}{\partial w_j} = (a - y)x_j \\ \hspace{15em} = -(y - a)x_j \end{array}$$

We take steps towards opposite direction of gradient, hence:

$$w_j := w_j + \eta(y - a)x_j$$

$$b := b + \eta(y - a)$$

Regularization for logistic regression

What is regularization?

Regularization is a technique used to prevent overfitting by adding a penalty term to the cost function. It discourages the model from relying too heavily on specific features or assigning excessively large weights to features, thereby improving the model's generalization ability.

Why do we need regularization?

1. Overfitting problem

- Logistic regression may fit the training data too well, capturing noise instead of meaningful patterns, especially when the dataset has many features or is small.

- Overfitting results in poor performance on unseen data.

2. Simpler models are preferable

- Regularization encourages simpler models by penalizing complex weight configurations.
- Simpler models often generalize better to new data.

3. Feature importance

- By shrinking some weights toward zero, regularization can highlight the most important features while ignoring less relevant ones.

L2 regularization

Here, we only introduce one of the most common forms of regularization, so called **L2 regularization**:

$$\frac{\lambda}{2n} \|\mathbf{w}\|^2 = \frac{\lambda}{2n} \sum_{j=1}^m w_j^2$$

Where,

- **λ : Regularization parameter** controls the strength of the penalty. Larger λ values impose a heavier penalty, resulting in smaller weights. L2 regularization encourages weights (w_j) to be small but does not force them to be exactly zero. Works well when most features are relevant to the task.
 - Small λ : Minimal regularization, allowing the model to closely fit the training data.
 - Large λ : Strong regularization, simplifying the model but potentially underfitting the data.
 - λ must be tuned, often using cross-validation (will be introduced in the later section).

Therefore, the cost function for logistic regression (we derived earlier) can be regularized by adding a simple regularization term:

$$L(\mathbf{w}, b) = L(\mathbf{w}, b) + \frac{\lambda}{2n} \|\mathbf{w}\|^2$$

We can expand it as:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \left[-y^{(i)} \log(\sigma(z^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(z^{(i)})) \right] + \frac{\lambda}{2n} \|\mathbf{w}\|^2$$

Let's only looking at the loss,

- The partial derivative of the unregularized loss:

$$\frac{\partial L(\mathbf{w}, b)}{\partial w_j} = \left(\frac{1}{n} \sum_{i=1}^n (\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right)$$

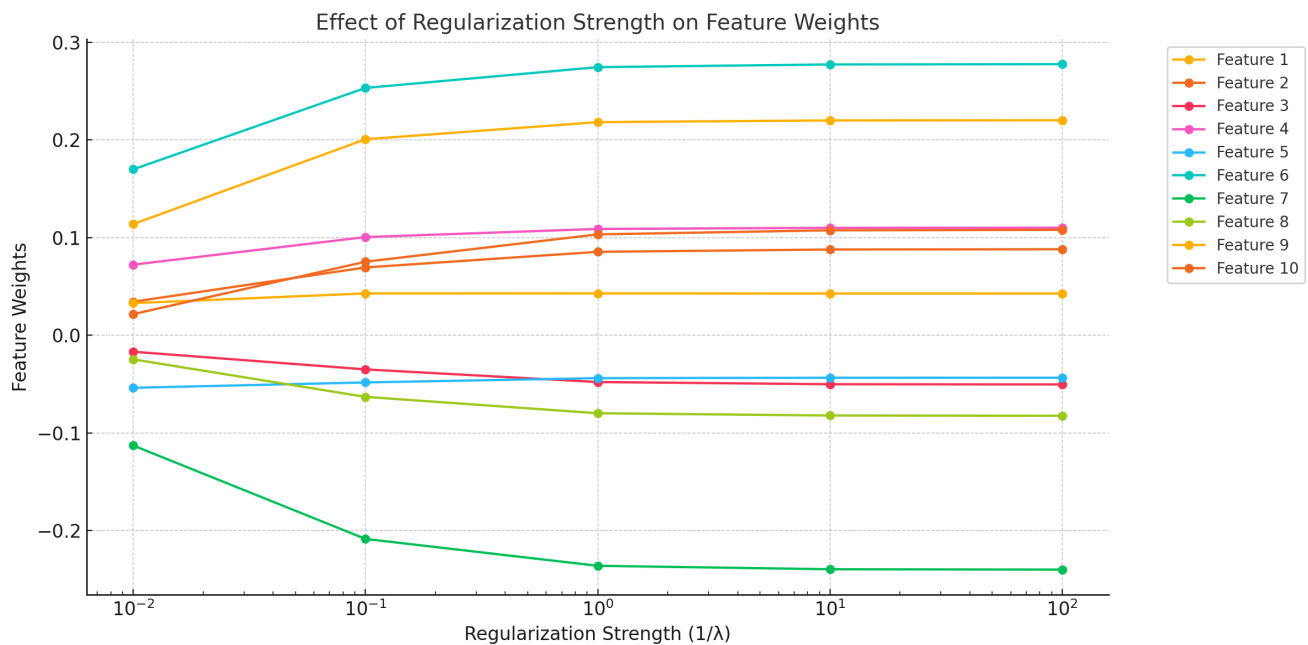
- The partial derivative of the regularized loss by adding λ :

$$\frac{\partial L(\mathbf{w}, b)}{\partial w_j} = \left(\frac{1}{n} \sum_{i=1}^n (\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{n} w_j$$

Visualization

Observations

1. As regularization strength decreases (λ increases), the weights shrink toward zero.
2. Features with less relevance to the classification task diminish faster, illustrating the role of regularization in controlling model complexity.



Kernel SVM for Nonlinear Problem

Overview

What is an SVM?

Support Vector Machines (SVM) are powerful supervised learning models used for classification and regression. They aim to find the optimal hyperplane that separates data points of different classes with the largest margin.

Key concepts

1. Linear SVM

- In the simplest case, SVM finds a straight line (or hyperplane in higher dimensions) to separate two classes in a dataset.
- It maximizes the margin, defined as the distance between the hyperplane and the nearest data points from each class (support vectors).

2. Nonlinear Data

- Many real-world datasets are not linearly separable. For such cases, SVM uses a **kernel trick** to project data into a higher-dimensional space where it becomes linearly separable.

Pros and cons of using SVM

Pros

1. Effective in high-dimensional spaces.

2. Works well when the number of features exceeds the number of samples.
3. Robust to overfitting, especially with proper kernel and regularization.

Cons

1. Choosing the right kernel and tuning its parameters can be challenging.
2. Computationally intensive for large datasets.
3. Performance may degrade if there is significant overlap between classes.

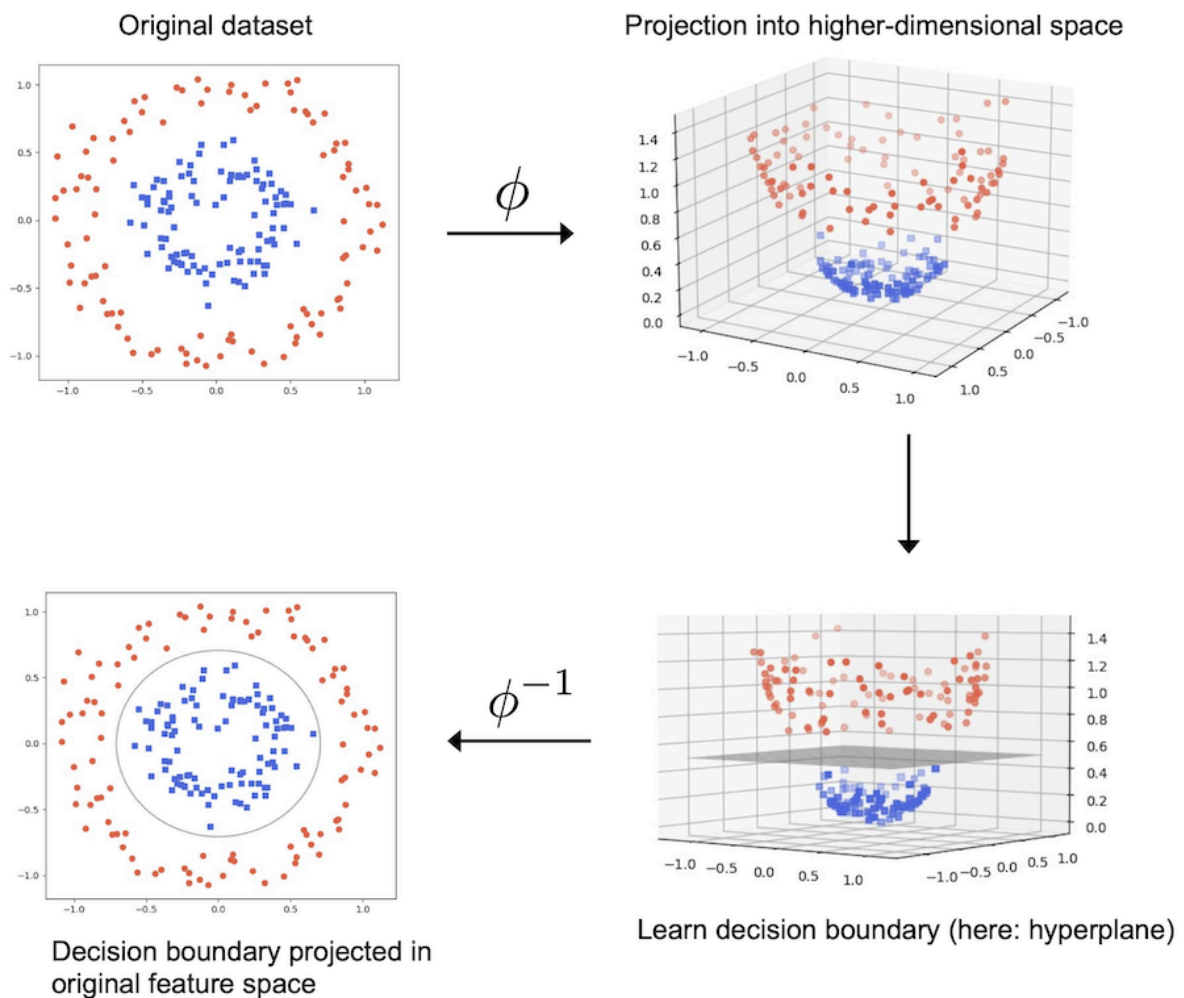
Kernel method in SVM

The main idea of kernel method is to create nonlinear combinations of the original features to project them onto a higher-dimensional space via a mapping function, ϕ , where the data becomes linearly separable.

For example, we can transform a two-dimensional dataset into a new three-dimensional feature space, where the classes become separable via the following projection:

$$\phi(x_1, x_2) = (z_1, z_2, z_3) = (x_1, x_2, x_1^2 + x_2^2)$$

This allows us to separate the two classes shown in the plot via a linear hyperplane that becomes a nonlinear decision boundary if we project it back onto the original feature space, ϕ^{-1} as illustrated with the following concentric circle dataset:



After applying kernel method on training data, which transform the data to a higher-dimensional feature space, then we may train a linear SVM model to classify data in this new feature space. However, this mapping approach is computationally very expensive, especially for high-dimensional data. We may use **kernel trick** to solve this problem. To save the expensive step of calculating this dot product between two points explicitly, we define a so-called **kernel function**, which replacing the dot product $\mathbf{x}^{(i)T} \mathbf{x}^{(j)}$ by $\phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(j)})$:

$$\kappa(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(j)})$$

One of the most widely used kernels is the **radial basis function (RBF)** kernel, which can simply be called the **Gaussian kernel**:

$$\kappa(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp\left(-\frac{\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2}{2\sigma^2}\right)$$

This kernel measures similarity between data points. The parameter σ controls the width of the kernel. It is often simplified to:

$$\kappa(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp\left(-\gamma \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2\right)$$

Here, $\gamma = \frac{1}{2\sigma^2}$ controls the influence of a single training example.

The kernel now "looks like" **similarity function** between a pair of examples. The minus sign inverts the distance measure into a similarity score. Due to the exponential term, the resulting similarity score will fall into a range between 1 (for exactly similar examples) and 0 (for very dissimilar examples).

Hyperparameters in SVM

1. Regularization Parameter (C):

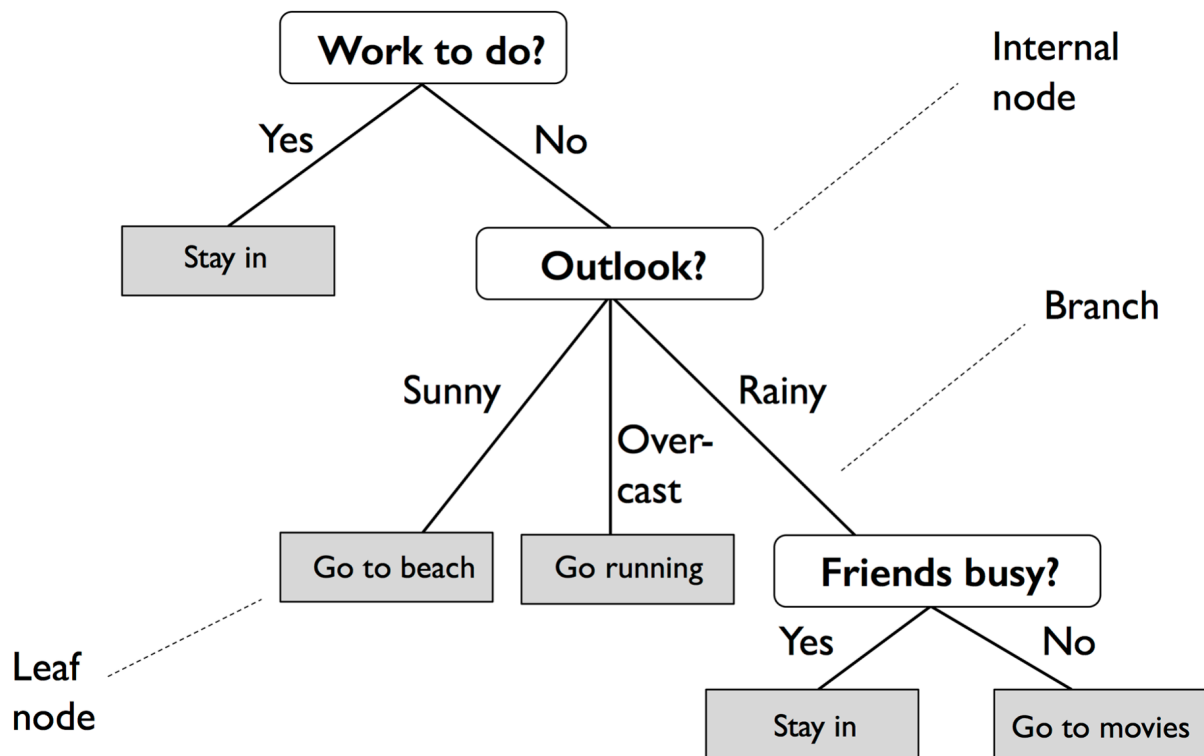
- Controls the trade-off between maximizing the margin and minimizing classification error.
- Small C : Allows for a larger margin, tolerating more misclassifications (simpler model).
- Large C : Focuses on minimizing errors, potentially leading to overfitting.

2. Kernel Parameters:

- For RBF kernel, σ (or $\gamma = \frac{1}{2\sigma^2}$) controls the influence of a single training example.
- Small σ : More flexible decision boundaries, higher risk of overfitting.
- Large σ : Smoother decision boundaries, higher risk of underfitting.

Now, we can tackle the nonlinear problem with kernel SVM model!

Decision Tree



Overview

A decision tree is a supervised learning algorithm that splits data into smaller subsets based on feature values to make predictions. It resembles a flowchart, where each internal node represents a decision rule, each branch represents an outcome, and each leaf node represents a final prediction. It sub-divides the input space into smaller regions that become more manageable, then analyze one feature at a time, rather than taking weighted combinations into account.

Key Concepts

1. Tree Structure:

- **Root Node:** The topmost node that represents the entire dataset.
- **Internal Nodes:** Represent decisions based on feature values.
- **Leaf Nodes:** Represent final outcomes or predictions.

2. Recursive Splitting:

- At each step, the tree splits data into subsets based on the feature and value that maximize a specific criterion, such as **information gain**; or that minimizes **impurity** or **entropy**. (We will introduce them in a minute.)

Algorithm

🔗 Decision Tree Algorithm

Start with all examples at the root node.

Calculate information gain for all possible features, and pick one with the highest information gain.

Split dataset according to selected feature, and create left and right branches of the tree.

Keep repeating splitting process until stopping criteria is met:

- When a node is 100% one class.

- When splitting a node will result in the tree exceeding a maximum depth.
- Information gain from additional splits is less than threshold.
- When number of examples in a node is below a threshold.

Pros and cons of using decision tree

Pros

1. Interpretability

- Easy to understand and visualize.
- Suitable for explaining decisions to non-technical audiences.

2. Flexibility

- Handles numerical and categorical features.
- Requires little to no preprocessing (e.g., no need for feature scaling).

3. Nonlinear relationships

- Captures complex patterns by splitting on different features at different levels.

Cons

1. Overfitting

- Trees can grow too deep, memorizing the training data instead of generalizing.
- Solution: Use techniques like pruning or setting a maximum depth.

2. Instability

- Small changes in the data can result in very different trees.
- Solution: Use ensembles like random forests or gradient boosting.

Improving decision trees

- **Pruning:**
 - Reduces overfitting by removing branches that add little predictive power.
- **Hyperparameter Tuning:**
 - Adjust parameters like `max_depth`, `min_samples_split`, and `min_samples_leaf`.
- **Ensemble Methods:**
 - Combine multiple trees using random forests or gradient boosting to improve accuracy and robustness.

Maximizing information gain

We grow a decision tree by iteratively splitting its nodes until the child nodes (leaves) are "pure" or a stopping criterion is satisfied.

When using decision trees for classification, we define **entropy** as a *measure of impurity or uncertainty in the data*:

$$I(p) = - \sum_i p_i \log_2 (p_i)$$

Also, we use **Information Gain (IG)** to quantify the reduction in entropy after a split, which also means that we use entropy to determine which feature split maximizes IG. It can be defined as follows for a binary split:

$$IG(D_p, f) = I(D_p) - \frac{N_{\text{left}}}{N_p} I(D_{\text{left}}) - \frac{N_{\text{right}}}{N_p} I(D_{\text{right}})$$

Where,

- f : The feature to perform the split.
- N_p : The number of training examples in the parent node.
- I : The impurity function.
- D_p : The subset of training examples at the parent node.
- D_{left} and D_{right} : The subsets of training examples at the left and right child nodes after the split.

The three impurity measures:

1. **entropy** (I_H): A criterion to maximize the mutual information in the tree. It measures the impurity or uncertainty in the data:

$$I_H = - \sum_i p_i \log_2(p_i)$$

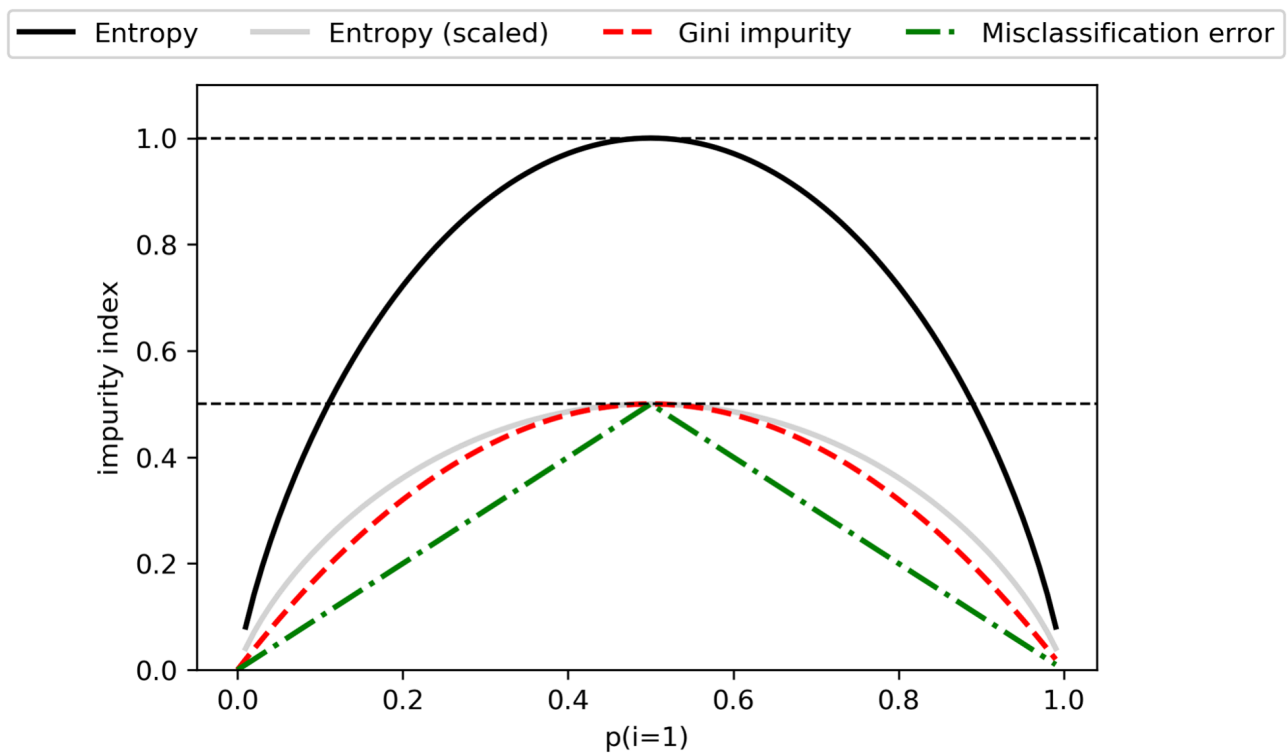
2. **Gini impurity** (I_G): A criterion to minimize the probability of misclassification. It measures the likelihood of incorrect classification if a random sample is chosen:

$$I_G = 1 - \sum_i p_i^2$$

3. **classification error** (I_E): A criterion for pruning, but not recommended for growing a decision tree, since it is less sensitive to changes in the class probabilities of the nodes. It is a simplest measure of impurity but less sensitive than entropy or Gini:

$$I_E = 1 - \max(p_i)$$

It is often not worth spending much time on evaluating trees using different impurity criteria rather than experimenting with different pruning cut-offs.



Example of decision tree

Consider a dataset for predicting whether someone buys a car based on Age and Income.

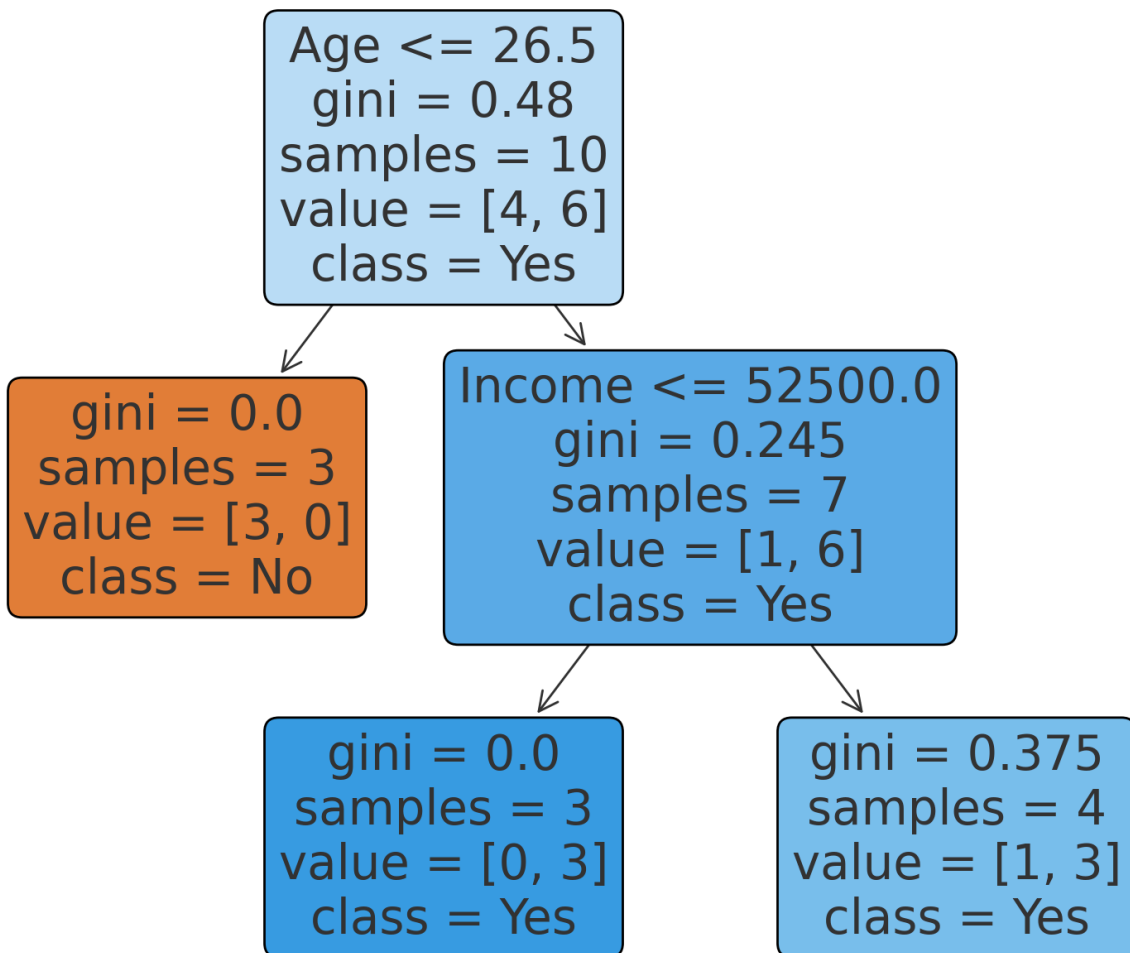
Step 1: Evaluate splits for Age (1st split) and Income (2nd split).

- Calculate Gini impurity for each possible split. Gini impurity value closer to 0 means the samples in the node mostly belong to one class. Higher values indicate a mix of classes.
- Choose the split that maximizes information gain.

Step 2: Continue splitting the data recursively.

Step 3: Stop splitting when stopping criteria are met (e.g., maximum depth reached or nodes are pure).

Decision Tree Visualization with Gini Impurity



Observations

- **1st Split (Age ≤ 26.5):**
 - `value=[4, 6]` : This means there are **4 "No" samples** (class 0) and **6 "Yes" samples** (class 1) in this node.
 - The predicted class for this node is "Yes" (class 1) because it is the majority class.
 - The Gini impurity value of `0.48` indicates moderate impurity since the samples are not perfectly separated (both classes are present).
 - **2nd Split (Income ≤ 52,500):**
 - `value=[1, 6]` : This means there are **1 "No" sample** and **6 "Yes" samples** in this node after splitting based on the current criterion.
 - The predicted class is still "Yes" (class 1), as the majority of samples belong to class 1.
 - The Gini impurity value of `0.245` is lower, reflecting a purer node due to the predominance of "Yes" samples.
 - **For the Rest of the Tree:**
 - The same logic applies recursively to each node. The `value` field indicates the distribution of "No" and "Yes" samples at each step, and the Gini impurity quantifies the degree of impurity within the node.
-

Bias-Variance Tradeoff

Key to the process of building machine learning system is how to decide what to do next in order to improve his performance. For many different applications that looking at the bias and variance of a learning algorithm gives you very good guidance on what to try next. The **bias-variance trade-off** is a fundamental concept in machine learning that explains the balance between a model's complexity and its ability to generalize to new data.

Variance

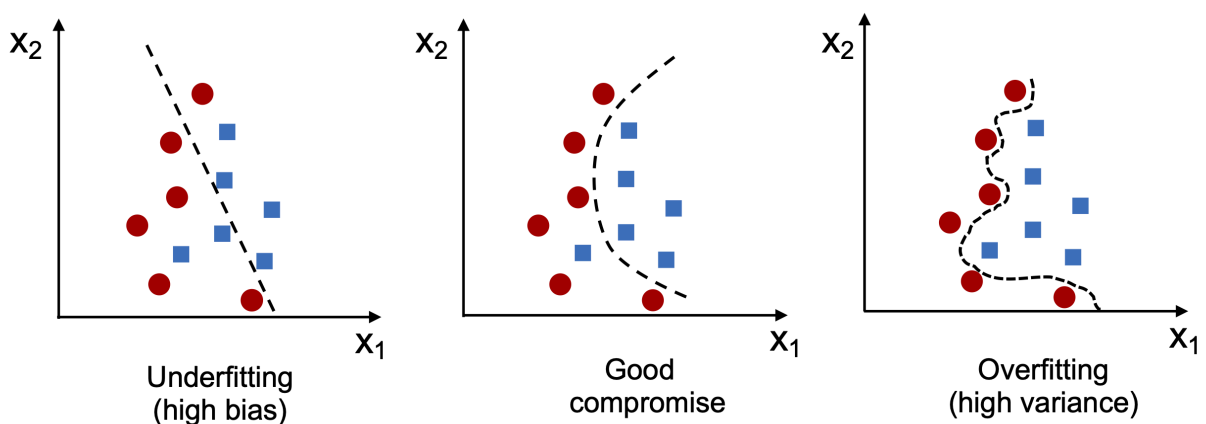
- Variance measures the consistency of model predictions for a particular example when retrained multiple times on different subsets of the training dataset.
- Variance is a measure of **sensitivity to randomness** in the training data.
- **High Variance**: The model is too complex and captures noise along with the underlying patterns. It leads to **overfitting**, where the model performs well on training data but poorly on test data. E.g., a decision tree with a very large depth will have high variance, as it memorizes the training data.

Bias

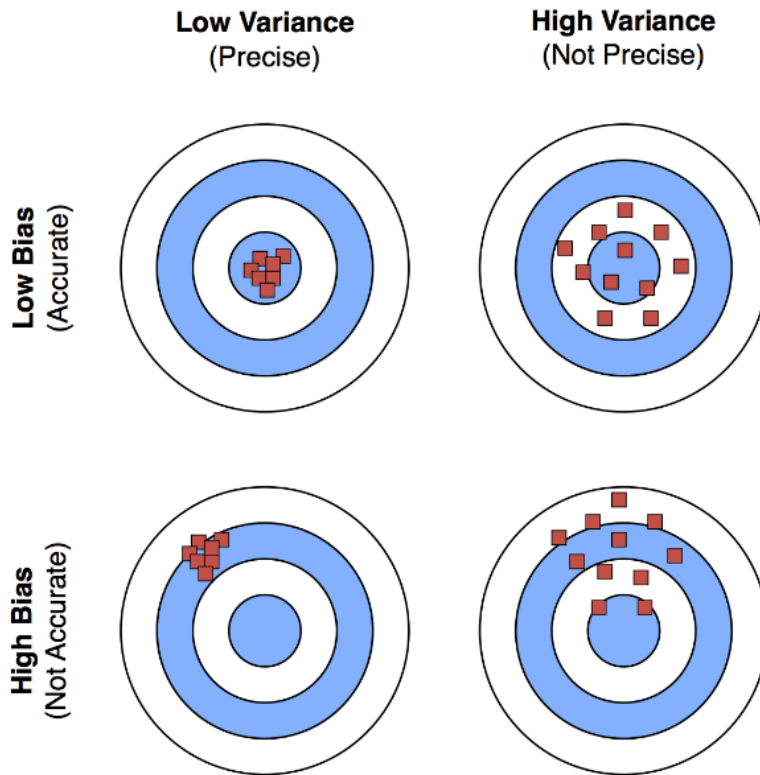
- Bias measures the difference between the model predictions and the correct values when retrained multiple times on different training datasets.
- Bias is a measure of **systematic error** not due to randomness.
- **High Bias**: The model is too simple to capture the underlying patterns in the data. It leads to **underfitting**, where the model performs poorly on both training and test datasets. E.g., a linear regression model trying to fit nonlinear data will have high bias because it oversimplifies the problem.

Some useful illustrations

- **Overfitting vs. Underfitting**



- **Bias vs. Variance**



Mathematical formulation

In regression tasks, we aim to find a function $f(x)$ that predicts the target variable t based on the input x . The true relationship between x and t is represented by the regression function:

$$h(\mathbf{x}) = \mathbb{E}[t | \mathbf{x}] = \int tp(t | \mathbf{x})dt$$

This is the expected value of t (the target) given x , meaning $h(x)$ represents the **true underlying function** we are trying to approximate.

The expected squared loss is a measure of how far the model's predictions $f(x)$ deviate from the true values t . It can be expressed as:

$$\mathbb{E}[L] = \int \{f(\mathbf{x}) - h(\mathbf{x})\}^2 p(\mathbf{x}) d\mathbf{x} + \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt$$

1. The first term, $\int \{f(x) - h(x)\}^2 p(x) dx$, depends on the model $f(x)$ and measures the error caused by its deviation from $h(x)$.
2. The second term, $\int \int \{h(x) - t\}^2 p(x, t) dx dt$, is the **irreducible noise** in the data, independent of $f(x)$.

As we cannot know the exact $h(x)$, for any given data set $\mathcal{D}$, we can run our learning algorithm and obtain a prediction function $f(x; \mathcal{D})$. To understand the first term ($\{f(x) - h(x)\}^2$), let's introduce an averaging process over multiple datasets $\mathcal{D}$ to measure how the function $f(x; \mathcal{D})$ varies.

1. Consider the squared error for a single dataset $\mathcal{D}$:

$$\{f(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2$$

2. Average this over all possible datasets $\mathcal{D}$, leading to:

$$\mathbb{E}_{\mathcal{D}} [\{f(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2]$$

3. The above can be decomposed as:

$$\begin{aligned} & \mathbb{E}_{\mathcal{D}} [\{f(\mathbf{x}; \mathcal{D}) - h(\mathbf{x})\}^2] \\ &= \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - h(\mathbf{x})\}^2}_{(\text{bias})^2} + \underbrace{\mathbb{E}_{\mathcal{D}} [\{f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})]\}^2]}_{\text{variance}}. \end{aligned}$$

Key components of expected loss

1. **Bias:**

- $\text{Bias}^2 = \{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})] - h(x)\}^2$
- Measures how much the average prediction $\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})]$ differs from the true function $h(x)$.
- High bias arises when the model is too simple to capture the true data structure (e.g., fitting a linear model to nonlinear data).

2. **Variance:**

- $\text{Variance} = \mathbb{E}_{\mathcal{D}} [\{f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})]\}^2]$
- Measures how much $f(x; \mathcal{D})$ fluctuates around its mean $\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})]$ for different datasets.
- High variance occurs when the model is too complex and overly sensitive to the training data (e.g., fitting a high-degree polynomial).

3. **Noise:**

- $\text{Noise} = \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t) d\mathbf{x} dt$
- Irreducible error due to inherent randomness in the data.

The total expected squared loss is the sum of the bias, variance, and noise terms:

$$\text{Expected Loss} = (\text{Bias})^2 + \text{Variance} + \text{Noise}$$

Example

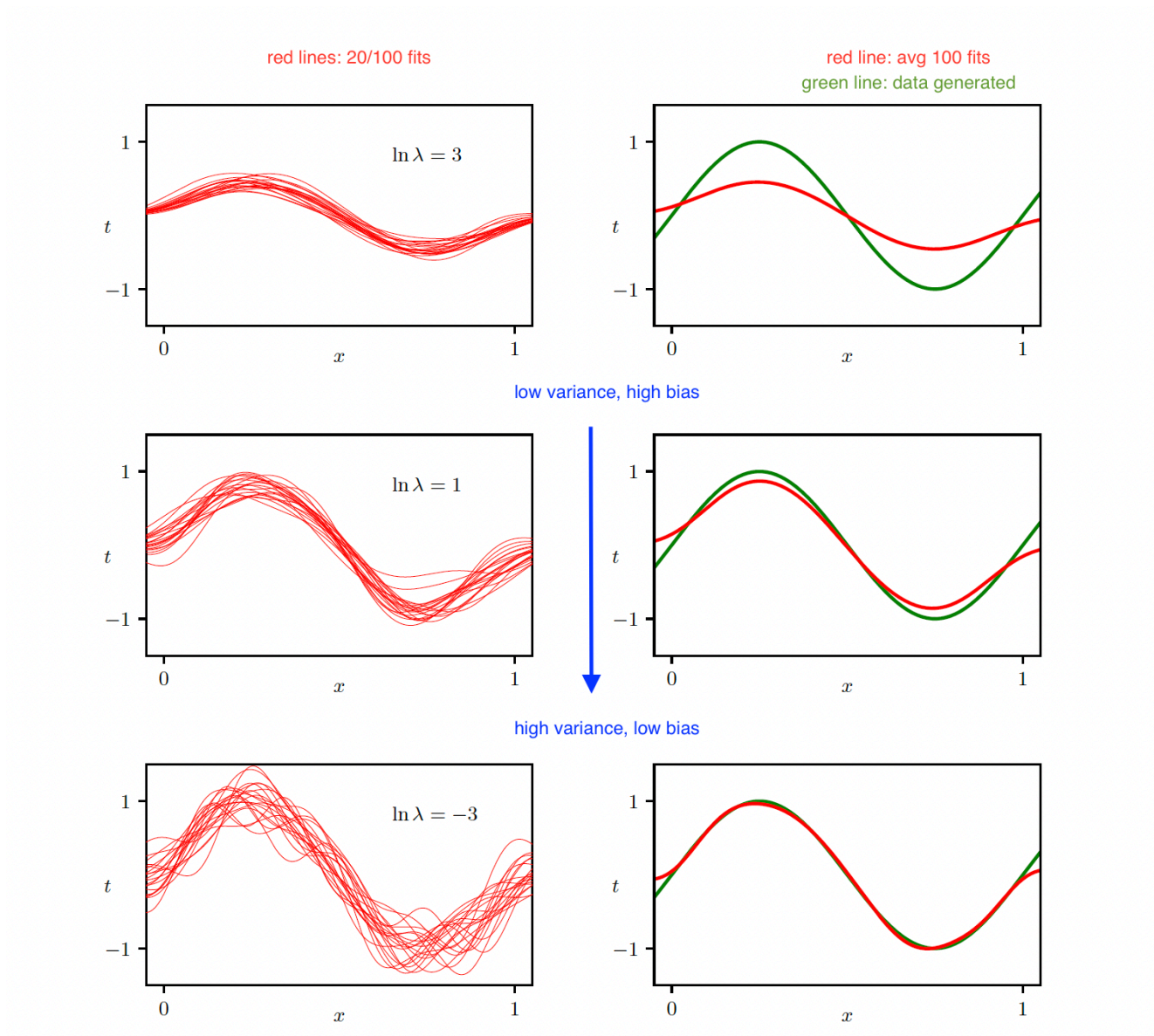
The sinusoidal dataset $h(x) = \sin(2\pi x)$ is used to illustrate the bias-variance trade-off:

1. For **large regularization (high λ)**:

- The model is simple, resulting in **low variance** (all model fits look similar).
- However, the predictions deviate significantly from the true function, leading to **high bias**.

2. For **small regularization (low λ)**:

- The model is highly flexible, resulting in **low bias** (good fit to the data).
- However, the fits vary significantly between datasets, leading to **high variance**.



Key takeaways

1. Trade-Off

- Complex models reduce bias but increase variance.
- Simple models reduce variance but increase bias.

2. Optimal Model

- Minimizes the sum of $(\text{Bias})^2 + \text{Variance}$, which aligns with minimizing total error.

3. Practical Insight

- In practice, balancing regularization (λ) or model complexity achieves the best trade-off.